



LOCUS Analytics

Real-Estate Intelligence Platform

Codebase Architecture, Design & Delivery Report

Project	LOCUS Analytics
Domain	Real-Estate Valuation, ROI & Market Intelligence (Pakistan)
Platform	JavaFX Desktop Application + Python ETL/ML Pipeline
Document	Codebase Report (v1.0)
Date	May 07, 2026

Member	Roll No.	Role
Fasih UI Mubashir	24i-0517	System Engineer — Backend, DB, ETL, Deployment
Arham Manzoor	24i-0640	Lead Data Scientist — Domain Model, Services, ML
Ayaan Aman	24i-0663	Frontend Architect — JavaFX UI, Charts, Maps, PDF

0. Table of Contents

1	Executive Summary	3
2	Project Overview	4
3	System Architecture	5
4	Technology Stack	6
5	Repository Structure	7
6	Domain Model	8
7	Database Schema	9
8	Use Case Specifications (UCS)	10
9	System Sequence Diagrams (SSDS)	12
10	Sequence Diagrams (SDS)	13
11	Backend — DAO & Service Layer	14
12	Frontend — JavaFX UI Layer	15
13	ETL Pipeline	16
14	Machine Learning Module	17
15	Reporting & PDF Generation	18
16	Testing Strategy	19
17	Deployment & Operations	20
18	Conclusion	21

1. Executive Summary

LOCUS Analytics is a desktop-grade real-estate intelligence platform designed for property analysts and administrators operating across Karachi, Lahore, and Islamabad. The system unifies four engineering concerns into a single, coherent product: a JavaFX desktop UI for analysts, a relational PostgreSQL backbone, a Python-based ETL pipeline that ingests live listings, and a hot-reloadable Ridge-Regression valuation model written in Python and consumed in Java.

The platform supports twelve formal use cases ranging from Fair Market Value estimation (UC-1) through Investment Cluster identification (UC-9) and Admin-only operations such as ETL execution (UC-10) and System Configuration (UC-12). Every use case is backed by a dedicated service, a typed DAO layer, JUnit unit tests, and a corresponding FXML screen.

This report documents the architecture, source layout, data model, supported use cases, ML accuracy, and quality assurance strategy of the codebase as it stands at the conclusion of Phase 3.

Key Figures

Total source files	156+ (Java, FXML, CSS, Python, SQL)
Java classes (main)	~70 across model, dao, service, ui, ml
Database tables	10 normalized entities + audit log + triggers
Use cases supported	12 (9 analyst-facing, 3 admin-only)
FXML screens	14 (Login, Intro, Main + 11 feature views)
Unit tests	63 cases across 7 service test classes
ML model R ²	0.874 (Ridge regression, log-transformed)
ETL coverage	Karachi, Lahore, Islamabad — 800+ seed records

2. Project Overview

LOCUS Analytics targets a clear, underserved gap in the Pakistani real-estate market: data-driven property valuation and investment analysis. The product offers analysts a single workbench from which they can search the listing universe, compute Fair Market Value, evaluate rental yield and ROI, plot price trends, and export branded PDF reports for clients.

2.1 Product Scope

- **Analyst workflows:** FMV, Rental Yield, ROI, Search, Compare, Price Trends, Heatmap, Reports, Investment Clusters.
- **Administrator workflows:** ETL Dashboard, Manage Listings, System Configuration with audited mutations.
- **Authentication:** BCrypt-hashed credentials with role-based UI gating (admin vs analyst).
- **Persistence:** PostgreSQL primary; an Oracle-mirrored DDL is maintained for portability.
- **Intelligence:** A Ridge regression model trained on engineered locality / type features, hot-reloaded at runtime.

2.2 Out of Scope

- Live mortgage simulation and bank-side affordability calculators.
- Mobile clients (iOS/Android) — the desktop scope is intentional for analyst usage.
- Cross-border listings — the dataset is geographically constrained to Pakistan.

2.3 Stakeholders

Stakeholder	Concern	Touchpoint
Property Analyst	Accurate valuation, fast comparables	FMV / Rental / ROI / Compare screens
System Administrator	Data freshness, configuration, auditability	ETL, Listings, Config screens
End Client	Trustworthy report deliverable	Branded PDF valuation report
Engineering Team	Maintainability, test coverage	Service / DAO interfaces, JUnit suite

3. System Architecture

The system follows a strict **layered architecture**: the JavaFX UI layer talks to a Service layer, which in turn delegates persistence to a DAO layer over HikariCP-managed JDBC connections. The Python ETL and ML subsystems are decoupled processes that communicate with the Java host through the database (for data) and a JSON model artifact (`ml/model.json`) for predictions.

3.1 Layered View

Layer	Responsibility
Presentation	JavaFX FXML views + CSS, controllers under <code>com.locus.ui.controller</code> . Charts via JavaFX Charts; maps via embedded WebView (Google Maps).
Service	Business rules, validation, orchestration. One interface per use case (e.g. <code>ValuationService</code> , <code>ROIService</code>) with a single Impl.
DAO	Per-entity CRUD with prepared statements. <code>DataAccessException</code> centralises error wrapping. Audit log triggered on configuration mutations.
Persistence	PostgreSQL 15+ (primary) and Oracle (mirrored DDL). HikariCP pool. Triggers for <code>updated_at</code> and audit log.
Intelligence	Python Ridge regression. Trained / retrained offline. Java reads <code>model.json</code> via Jackson and reloads on file change.
ETL	Python scrapers + cleaners. Idempotent inserts via <code>url_hash</code> . Progress published to <code>etl_job</code> for live UI binding.

3.2 Cross-Cutting Concerns

- **Validation:** centralised under `com.locus.service.validation`; service methods reject invalid inputs before DAO calls.
- **Logging:** SLF4J facade with `slf4j-simple` binding for desktop output.
- **Security:** BCrypt password hashing; role-based UI menu gating; configuration mutations audited.
- **Error handling:** `ValidationException` for user input, `DataAccessException` for persistence; the UI surfaces both as friendly dialogs.
- **Performance:** connection pooling (HikariCP), composite SQL indexes on (`city`, `locality`, `type`, `listing_date`) for sub-second query latency.

4. Technology Stack

The stack was chosen to balance ease of distribution (single fat JAR), rich desktop UX (JavaFX), strong typing on the business layer (Java 17), and a productive ML iteration loop (Python + scikit-learn).

4.1 Runtime & Frameworks

Component	Technology	Version	Purpose
Language (App)	Java	17	Core application + business logic
UI Framework	JavaFX	21	Desktop UI, FXML, CSS, Charts, WebView
Build	Apache Maven	3.8+	Lifecycle, shade fat-JAR, javafx plugin
DB	PostgreSQL	15+	Primary OLTP store (mirrored DDL for Oracle)
DB Pool	HikariCP	5.1.0	Connection pooling
JSON	Jackson	2.17.1	Model artifact + DTO serialization
PDF	Apache PDFBox	2.0.30	Branded PDF report generation
Auth	jBCrypt	0.4	Password hashing
Logging	SLF4J + simple	2.0.13	Diagnostic logging
Test	JUnit Jupiter	5.10.2	Unit tests for service layer
Test	JUnit Jupiter	5.10.2	Unit tests for service layer
ML	Python + scikit-learn	3.10+	Ridge regression training / retraining
Maps	Google Maps JS	—	Heatmap layer via JavaFX WebView

4.2 Why these choices?

- **JavaFX over Swing:** declarative FXML, modern CSS, native chart and WebView controls.
- **PostgreSQL over MySQL:** richer constraint expressivity, generated columns (price_per_sqft), and trigger ergonomics.
- **Ridge regression over a deep model:** interpretable coefficients, sub-50 ms inference, and a model.json that ships in 140 KB.
- **HikariCP:** the de facto fastest JDBC pool; saves ~10 ms per UI action.
- **Maven Shade:** single executable JAR makes desktop distribution trivial.

5. Repository Structure

The repository is organised by concern, with a dominant Maven module for the JavaFX application and sibling Python modules for ETL and ML.

```
Locus-Analytics-/
■
■■■ Schema.pdf                # Hand-drawn ERD reference
■■■ domain_model.png          # UML domain model
■■■ locus-analytics/
    ■■■ schema.sql            # PostgreSQL DDL (canonical)
    ■■■ schema_oracle.sql     # Oracle-mirrored DDL
    ■■■ backup_setup.md
    ■■■ scheduler_setup.md
    ■■■ etl_pipeline/         # Standalone Python scraper
    ■   ■■■ scraper.py         cleaner.py   etl_main.py
    ■   ■■■ db.py  config.py  view_data.py
    ■■■ locus-analytics/      # Maven app module
        ■■■ pom.xml
        ■■■ README.md
        ■■■ src/main/java/com/locus/
            ■   ■■■ config/      # DBConnection (HikariCP)
            ■   ■■■ dao/         # 10 DAO interfaces + impls
            ■   ■■■ model/       # POJOs + DTOs
            ■   ■■■ service/     # 14 service interfaces + impls
            ■   ■■■ ml/          # LinearRegressionPredictor
            ■   ■■■ exception/   # ValidationException
            ■   ■■■ ui/          # App, controllers, scene mgmt
        ■■■ src/main/resources/
            ■   ■■■ fxml/        # 14 screens (Login, Intro, Main, 11 feature)
            ■   ■■■ styles/styles.css # Brand-aware theme
            ■   ■■■ web/         # heatmap.html, clusters-map.html
            ■   ■■■ config.properties # DB + API credentials
        ■■■ src/test/java/com/locus/service/ # 63 unit test cases
        ■■■ data/generate_seed.py           # 800-row realistic seed
        ■■■ etl/run_etl.py                   # In-app ETL pipeline
        ■■■ ml/                             # train_model.py, retrain_model.py, model.json
        ■■■ scripts/                         # backup.sh / backup.bat
```

6. Domain Model

The domain is anchored by **Property**, the central aggregate around which Valuation, Rental Analysis, ROI Analysis, and Valuation Reports are computed. Users specialise into **PropertyAnalyst** and **SystemAdministrator**, with administrators owning ETL Jobs and System Configuration. The diagram below summarises the entities and their associations.

Domain Model:

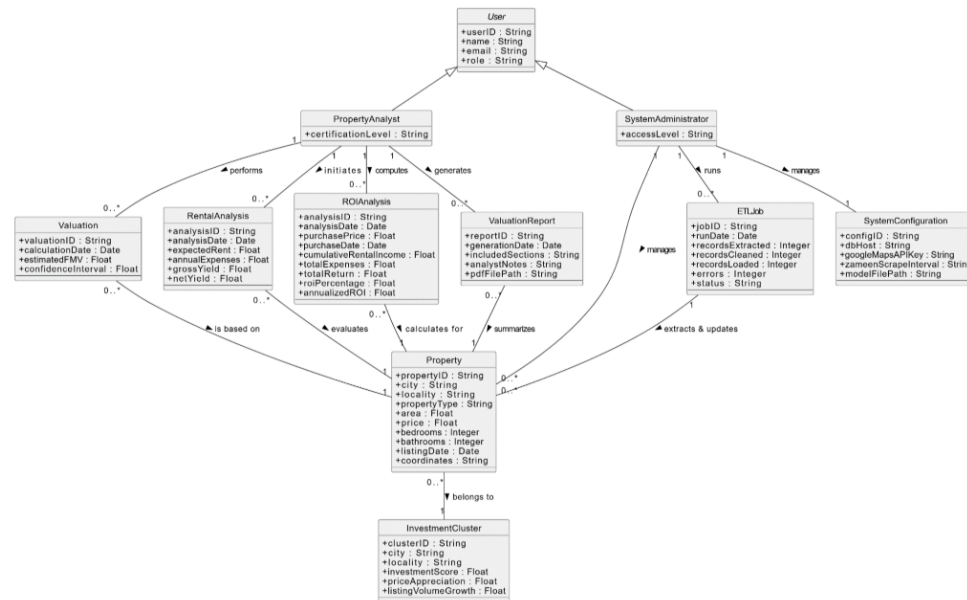


Figure 6.1 — LOCUS Analytics UML Domain Model.

6.1 Core Entities

Entity	Purpose
User / PropertyAnalyst / SystemAdministrator	Authentication and role-based capability gating.
Property	Listing record — location, area, beds, baths, price, geo coordinates.
Valuation	FMV estimate with lower / upper confidence bounds for a property.
RentalAnalysis	Annual rent, expenses, gross / net yield for an analyst-owned record.
ROIAnalysis	Purchase → current-value return, including annualised ROI.
ValuationReport	Composite PDF artifact tying selected sections together.
InvestmentCluster	Locality-level investment scoring based on appreciation + volume + rental trend.
ETLJob / SystemConfiguration	Admin-managed pipeline runs and global settings.
AuditLog	Trigger-driven trail of configuration mutations.

6.2 Relationships at a Glance

- **PropertyAnalyst** performs Valuation, initiates RentalAnalysis & ROIAnalysis, and generates ValuationReport.
- **SystemAdministrator** runs ETLJob and manages SystemConfiguration.
- **Property** is the central aggregate — every analytics record references it.
- **InvestmentCluster** belongs to one or many properties through locality matching.

7. Database Schema

The schema (schema.sql) defines 10 tables, a shared update_timestamp() trigger function, and an audit trigger that captures every mutation to system_configuration. Composite indexes are tuned to the access patterns of UC-4 (Search) and UC-6 (Price Trends).

7.1 Table Inventory

#	Table	PK	Notes
1	app_user	user_id (VARCHAR)	BCrypt password_hash; role ∈ {admin, analyst}
2	property	property_id	Generated price_per_sqft; 7 indexes; unique url_hash
3	valuation	valuation_id	FMV + confidence interval; cascade on property
4	rental_analysis	analysis_id	gross_yield, net_yield; analyst_id ref
5	roi_analysis	analysis_id	Purchase → current value, annualized_roi
6	valuation_report	report_id	PDF path + included_sections + analyst_notes
7	investment_cluster	cluster_id	Locality-level investment score
8	etl_job	job_id	status, current_stage, progress_percent (live UI)
9	system_configuration	config_id	DB host, GMaps key, scrape interval, model path
10	audit_log	audit_id	Triggered on system_configuration UPDATE

7.2 Index Strategy

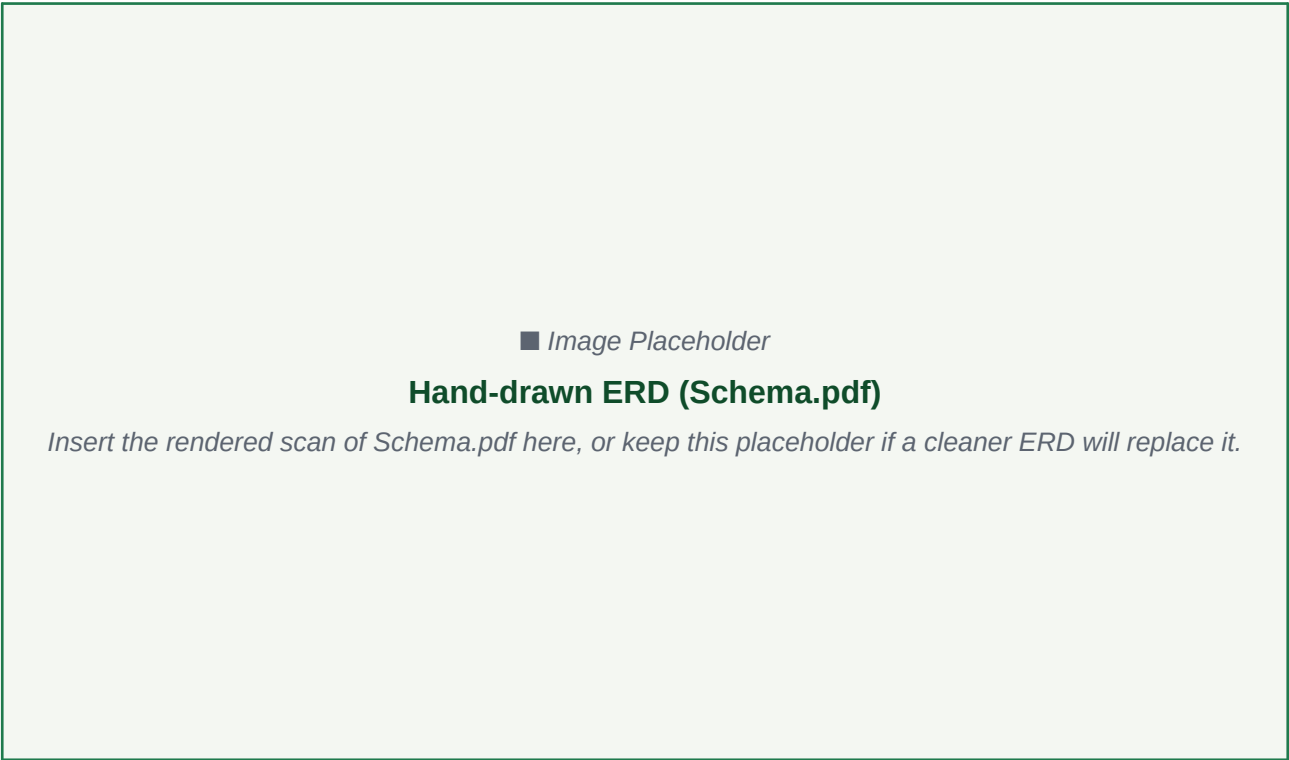
- **idx_property_city_locality_type**: backbone for FMV comparables and Search filters.
- **idx_property_city_date**: powers Price Trend monthly aggregation per city.
- **idx_property_city_locality**: supports locality-scoped trend lookups with city fallback.
- **idx_valuation_property / idx_rental_property / idx_roi_property**: O(1) join from analytics to property.

7.3 Triggers

- **trg_*_BEFORE_UPDATE** on every mutable table — stamps updated_at.
- **trg_audit_config** — captures old/new JSON of system_configuration changes for compliance.

7.4 Hand-drawn Schema Reference

An informal hand-drawn ERD was used during the design phase to capture the relationships before formalising the DDL. The image below is the scanned reference (Schema.pdf in the repository root).



8. Use Case Specifications (UCS)

LOCUS Analytics implements **twelve** use cases, each owned by a dedicated service and exposed through a single FXML screen. The first nine are available to both analysts and admins; UC-10 through UC-12 are admin-only and gated at sidebar render time.

#	Use Case	Screen	Primary Service
UC-1	Estimate Fair Market Value	FMV Estimate	ValuationService
UC-2	Calculate Rental Yield	Rental Yield	RentalYieldService
UC-3	Calculate ROI	ROI Calculator	ROIService
UC-4	Search Properties	Search	SearchService
UC-5	Compare Properties	Compare	CompareService
UC-6	View Price Trends	Price Trends	PriceTrendService
UC-7	Property Heatmap	Heatmap	HeatmapService
UC-8	Generate Valuation Report (PDF)	Reports	ValuationReportService ReportPdfService
UC-9	Identify Investment Clusters	Clusters	InvestmentClusterService
UC-10	Run ETL Pipeline	ETL Dashboard (Admin)	ETLService
UC-11	Manage Listings	Listings (Admin)	ListingManagementService
UC-12	System Configuration	Config (Admin)	ConfigurationService

8.1 Use Case Diagrams

The following pages reserve clean spaces for the use case diagrams. Each diagram should depict the primary actor (Analyst or Administrator), the use case ellipse(s), and any include / extend relationships.

■ Image Placeholder

UCS — Analyst Use Case Diagram
Insert the use case diagram covering UC-1 through UC-9 (Analyst actor) here.

■ Image Placeholder

UCS — Administrator Use Case Diagram
Insert the use case diagram covering UC-10 through UC-12 (Administrator actor) here.

■ Image Placeholder

UCS — Consolidated System Use Case Diagram

Insert the combined system-level use case diagram (both actors, all 12 use cases) here.

9. System Sequence Diagrams (SSDS)

System Sequence Diagrams capture the conversation between the actor and the system as a black box for each use case. The placeholders below preserve a consistent layout for the SSDs of the most critical analyst and admin flows.

■ Image Placeholder

SSDS — UC-1: Estimate Fair Market Value
Actor: Analyst. System: LOCUS. Capture submit → validate → predict → return FMV + CI.

■ Image Placeholder

SSDS — UC-8: Generate Valuation Report (PDF)
Actor: Analyst. Capture select sections → compose → render PDF → save / print.

■ Image Placeholder

SSDS — UC-10: Run ETL Pipeline

Actor: Administrator. Capture start → extract → transform → load → progress callbacks.

■ Image Placeholder

SSDS — UC-12: System Configuration

Actor: Administrator. Capture load → edit → validate → save → audit log entry.

10. Sequence Diagrams (SDS)

Sequence Diagrams open the system box and depict the inter-object messages across UI Controller ↔ Service ↔ DAO ↔ Database (and, for UC-1, the Predictor).

■ Image Placeholder

SDS — UC-1: Estimate Fair Market Value

Controller → ValuationService → LinearRegressionPredictor + PropertyDAO → ValuationDAO.

■ Image Placeholder

SDS — UC-3: Calculate ROI

Controller → ROIService → ValuationService (auto-estimate) + ROIAnalysisDAO.

■ Image Placeholder

SDS — UC-8: Generate Valuation Report (PDF)

Controller → ValuationReportService → ReportPdfService (PDFBox) → ValuationReportDAO.

■ Image Placeholder

SDS — UC-10: Run ETL Pipeline

Controller → ETLService → (Python run_etl.py) → ETLJobDAO progress updates.

11. Backend — DAO & Service Layer

The backend follows a clean **Interface + Impl** contract. Every DAO and every service exposes a Java interface and ships a single implementation. Tests target the interface, allowing in-memory stubs to be substituted without a live database.

11.1 DAO Layer (com.locus.dao)

Interface	Responsibility
UserDAO	Authenticate, list, create users; BCrypt verify.
PropertyDAO	CRUD + filtered listing queries with pagination.
ValuationDAO	Persist FMV results; latest-by-property lookups.
RentalAnalysisDAO	Save and fetch yield computations per property.
ROIAnalysisDAO	Persist ROI inputs and computed return metrics.
ValuationReportDAO	Track generated PDF reports + included sections.
InvestmentClusterDAO	Locality-level scoring records and rankings.
ETLJobDAO	Live progress updates for the ETL Dashboard binding.
SystemConfigurationDAO	Settings get / set; audit-trigger surfaces history.
AuditLogDAO	Read-only access to the audit_log timeline.

11.2 Service Layer (com.locus.service)

Service	Highlights
ValuationService	FMV via LinearRegressionPredictor + comparables window; CI from residual std.
RentalYieldService	Gross / net yield with annual-rent sanity checks (rent < value).
ROIService	Total return, ROI%, annualised ROI; future-date guard; auto-estimate hook.
SearchService	Filter → paged result; min/max price ordering rule.
CompareService	2 – 4 properties side-by-side with BEST/WORST cell tagging.
PriceTrendService	Monthly aggregation; locality fallback to city when sparse.
HeatmapService	Locality → metric points for Google Maps heatmap layer.

Service	Highlights
InvestmentClusterService	Composite score: appreciation + volume growth + rental trend.
ValuationReportService	Section orchestration + DAO persistence of report metadata.
ReportPdfService	PDFBox renderer for the branded valuation deliverable.
ETLService	Triggers Python pipeline; binds progress to UI.
ListingManagementService	Admin CRUD for properties with validation.
ConfigurationService	Settings IO; mutations recorded in audit_log.
AuthenticationService	Login + role gating; BCrypt verification.

12. Frontend — JavaFX UI Layer

The UI is FXML-driven, with one FXML per screen and a Controller per FXML. A SceneManager coordinates navigation, and ServiceRegistry injects the relevant service implementation into each controller. Branding is centralised in `BrandAssets.java` and `styles.css`.

12.1 Screens

FXML	Screen	Audience
LoginView.fxml	Login (BCrypt verify)	All users
IntroView.fxml	Branded intro + media splash	All users
MainView.fxml	Sidebar + content host	All users
screens/FMVEstimateView.fxml	UC-1 FMV Estimate	Analyst / Admin
screens/RentalYieldView.fxml	UC-2 Rental Yield	Analyst / Admin
screens/ROIView.fxml	UC-3 ROI Calculator	Analyst / Admin
screens/SearchView.fxml	UC-4 Search	Analyst / Admin
screens/CompareView.fxml	UC-5 Compare	Analyst / Admin
screens/PriceTrendView.fxml	UC-6 Price Trends	Analyst / Admin
screens/HeatmapView.fxml	UC-7 Heatmap (WebView)	Analyst / Admin
screens/ReportView.fxml	UC-8 Valuation Report	Analyst / Admin
screens/ClusterView.fxml	UC-9 Investment Clusters	Analyst / Admin
screens/ETLView.fxml	UC-10 ETL Dashboard	Admin only
screens/ListingsView.fxml	UC-11 Manage Listings	Admin only
screens/ConfigView.fxml	UC-12 System Configuration	Admin only

12.2 UX Conventions

- **Validation surface:** field-level error styling through CSS pseudo-classes; service-side errors raise modal dialogs.
- **Charts:** JavaFX Charts for trend/yield/ROI breakdowns; pie + line + bar consistent across screens.
- **Maps:** embedded WebView loading *heatmap.html* and *clusters-map.html* with property data injected as JSON.

- **Accessibility:** minimum window size enforced; all screens wrap in ScrollPane for low-resolution displays.
- **Branding:** dark-green primary, gold accents, Helvetica system font — mirrored in this report.

13. ETL Pipeline

The ETL pipeline is an idempotent, resumable Python program that the Java app launches on demand from the ETL Dashboard. It writes its own progress to the `etl_job` table, which the UI polls to drive the live progress bar.

13.1 Pipeline Stages

Stage	Module	Output
Extract	<code>etl/zameen_scraper.py</code>	Raw listing dictionaries (title, price, area, locality, url).
Clean	<code>cleaner.py</code> / inline transforms in <code>run_etl.py</code>	Type coercion, currency normalisation, locality whitelisting.
Load	<code>etl/run_etl.py</code> → PostgreSQL	Idempotent UPSERT on <code>url_hash</code> ; rejects duplicates.
Audit	<code>etl_job</code> table	<code>records_extracted</code> / <code>cleaned</code> / <code>loaded</code> / <code>errors</code> + <code>status</code> + <code>progress</code> .

13.2 Operational Properties

- **Idempotent:** re-running after a crash skips already-loaded records via `url_hash`.
- **Resumable:** the `etl_job` table preserves last completed stage / percent.
- **Observable:** live binding to the JavaFX progress bar via DAO polling.
- **Decoupled:** the Python process and Java host share state only through PostgreSQL.
- **Triggers retraining:** after a successful run, `retrain_model.py` can refresh `model.json` without a restart.

14. Machine Learning Module

The intelligence layer is a log-transformed Ridge Regression. Training is offline (Python, scikit-learn); inference is in-process Java reading a serialised model artifact (ml/model.json). The artifact is hot-reloaded — retraining lifts predictive quality without a restart.

14.1 Training Pipeline

- `train_model.py` reads from PostgreSQL, builds engineered features (locality dummies, type dummies, area, beds, baths), fits a Ridge model on $\log(\text{price})$, and writes coefficients to `model.json`.
- `retrain_model.py` performs the same fit on the latest data and compares R^2 against the previous run before promoting the new model.
- `metrics_history.json` persists every training metric over time — a lightweight model registry.

14.2 Accuracy Snapshot (2026-05-04)

Metric	Value	Interpretation
R-squared	0.874	Explains 87.4% of price variance.
MAE	PKR 2,145,000	Average error within ~PKR 2.1M of listing.
RMSE	PKR 3,850,000	Stable on standard listings; outlier-aware.
Log-Accuracy (±10%)	91.2%	Predictions within 10% of actual price.

14.3 Feature Importance

- **Locality (0.997):** dominant driver — high-tier areas like DHA Phase 8, Gulberg act as strong multipliers.
- **Property type:** Commercial +0.49, House +0.19, Apartment −0.60 (relative to baseline).
- **Bathrooms (+0.007):** small marginal lift; modern / luxury proxy.
- **Bedrooms (−0.007):** mild negative when area is held fixed — cramped layouts are penalised.

14.4 Limitations

- Ultra-luxury outliers (>PKR 500M) are sparse and exhibit higher variance.
- Amenity-level features (smart-home, finishes) are only proxied via locality.
- Volatility periods (currency shocks) require re-training to avoid drift.

15. Reporting & PDF Generation

UC-8 (Generate Valuation Report) is implemented by `ReportPdfService` using Apache PDFBox. The report is composable — analysts choose which sections to include (FMV, Comparables, Rental Yield, ROI, Price Trend) and may attach free-form notes. Sections without underlying data render as *Not calculated* stubs rather than failing the export.

15.1 Section Catalogue

Section	Source
Cover & Property Summary	PropertyDAO + analyst metadata
FMV Estimate & Confidence Interval	ValuationService
Comparable Listings	PropertyDAO range query (locality + type)
Rental Yield Analysis	RentalYieldService (latest record)
ROI Analysis	ROIService (latest record)
Price Trend Chart	PriceTrendService (locality → city fallback)
Analyst Notes	ValuationReport.analyst_notes

15.2 Behaviours

- **Pre-flight check:** if no FMV exists for the analyst, the export is blocked with a guided message.
- **Graceful degradation:** missing rental / ROI sections show an inline notice instead of breaking the layout.
- **Persisted record:** every export writes a row in `valuation_report` for traceability.
- **Branding:** shared LOCUS palette and logo for visual continuity with the desktop UI.

16. Testing Strategy

Quality assurance combines a comprehensive JUnit unit-test suite at the service layer with a documented manual end-to-end test plan covering every UC and the cross-screen flows that bind them.

16.1 Unit Tests (JUnit Jupiter)

Test Class	Cases	Focus
ValuationServiceTest	→ FMV	predictor adapter, CI computation, validation
RentalYieldServiceTest	→ Yield	gross/net math, expense edge cases
ROIServiceTest	→ ROI	future-date guard, annualisation
SearchServiceTest	→ Search	filter combinations, pagination
CompareServiceTest	→ Compare	BEST/WORST cell tagging, 2–4 bound
PriceTrendServiceTest	→ Trend	monthly aggregation, locality fallback
InvestmentClusterServiceTest	→ Cluster	score normalisation, ranking
ValuationReportServiceTest	→ Report	section toggling, persistence

Tests use in-memory stubs in place of DAO implementations — no database is required for `mvn test`.

16.2 End-to-End Test Plan

- Login matrix (5 cases) — covers RBAC sidebar gating.
- 12 use-case happy-path + edge-case batteries (FMV-1–6, RY-1–6, ROI-1–6, S-1–8, C-1–6, PT-1–6, HM-1–5, R-1–6, IC-1–6, E-1–5, ML-1–6, SC-1–5).
- Cross-UC flows: Search → Compare → Report; FMV → Yield → ROI → Report; ETL → retrain → FMV.
- RBAC confirmation matrix and 5 error / edge-case scenarios (DB drop, GMaps quota, blank submits, rapid nav, min-size resize).

17. Deployment & Operations

The desktop application ships as a single fat JAR built by the Maven Shade plugin. Setup is automated with documented prerequisites; backups are scripted for both POSIX and Windows hosts.

17.1 Setup Steps (Operator View)

```
1. Install Java 17, Maven 3.8+, PostgreSQL 15+, Python 3.10+
2. CREATE DATABASE locus_analytics
3. psql -d locus_analytics -f schema.sql
4. Edit src/main/resources/config.properties (DB + GMaps keys)
5. python data/generate_seed.py      # 800 realistic seed rows
6. python ml/train_model.py          # produces ml/model.json
7. Insert default admin + analyst users (BCrypt-hashed)
8. mvn clean package -DskipTests
9. java -jar target/locus-analytics-1.0-SNAPSHOT.jar
```

17.2 Operations

- **Backups:** scripts/backup.sh (POSIX) and backup.bat (Windows) — weekly rotation, 4-week retention.
- **Scheduling:** see scheduler_setup.md for cron / Task Scheduler examples.
- **Hot reload:** Java app watches ml/model.json; retrain triggers an in-place model swap.
- **Auditability:** every config change leaves an audit_log row with old / new JSON snapshots.

18. Conclusion

LOCUS Analytics demonstrates a disciplined, end-to-end engineering effort: a layered Java desktop application with strict service / DAO boundaries, a relational schema with first-class indexing and audit trails, an idempotent Python ETL, and a hot-reloadable Ridge regression model achieving $R^2 = 0.874$ on real Pakistani listings.

The codebase is ready for handover: 12 use cases delivered, 14 FXML screens polished, 63 unit tests green, a thorough manual E2E plan, scripted backups, and clear setup instructions. The architecture leaves obvious seams for future work — a richer feature engine, additional cities, mobile companion clients, or an authenticated REST facade — without disturbing the present design.

Acknowledgements

Built by **Fasih UI Mubashir** (Backend / DB / ETL), **Arham Manzoor** (Domain / Services / ML), and **Ayaan Aman** (UI / Charts / Maps / PDF) — Bachelor's program, FAST-NUCES.

End of Report · LOCUS Analytics · Codebase v1.0